ENGENHARIA DE SOFTWARE

Unidade 2: Modelos de Processos de Software

Marcos Antonio Quezado Soares, MSc

Material acadêmico



SUMÁRIO

Unidade 2: Modelos de Processos de Software

Três abordagens para compreender a evolução dos processos de desenvolvimento.

1

Modelo Iterativo e Incremental

Evolução por ciclos e entregas progressivas.

2

Modelo em Espiral

Iterações orientadas pela análise e redução de riscos.

3

RUP e UML

Fases, disciplinas e modelos visuais do sistema.

Objetivos de aprendizagem

Ao final da aula, o estudante deverá ser capaz de:

01 **Diferenciar** iteração de incremento.

02 **Explicar** como os conceitos se combinam.

03 **Avaliar** benefícios, limitações e riscos.

04 **Relacionar** espiral, UP e RUP.

05 **Selecionar** um modelo para cada cenário.

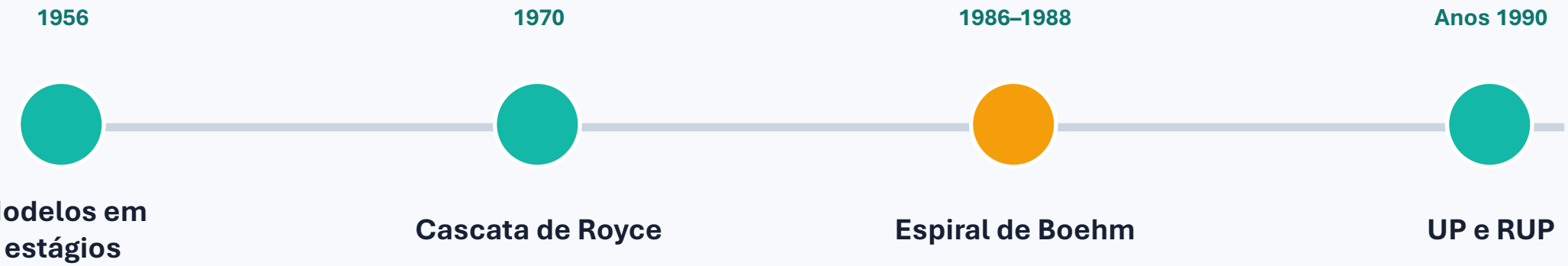
A pergunta-guia

Como entregar valor mais cedo sem perder qualidade, arquitetura e controle de riscos?

02 | HISTÓRICO

Da sequência rígida à decisão orientada a riscos

O espiral responde aos limites de processos estritamente lineares.



Mudança central: “qual risco deve ser reduzido agora?”

01

Modelo Iterativo e Incremental

Conceitos, características, vantagens e desvantagens



Marcos Antonio Quezado Soares | CEUB

Características do modelo

Entregas parciais

Versões utilizáveis ao longo do projeto.

Ciclos repetidos

Atividades de engenharia em cada iteração.

Feedback contínuo

Avaliação desde as primeiras versões.

Priorização

Valor, risco e dependências orientam a sequência.

Integração progressiva

Cada parte funciona com as anteriores.

Planejamento adaptativo

O plano é revisto com novas evidências.

O modelo combina repetição e crescimento progressivo

Iterativo

Desenvolvimento em ciclos sucessivos.
Cada ciclo revisa, testa e aperfeiçoa a solução.
O aprendizado orienta o ciclo seguinte.

Incremental

Desenvolvimento em partes funcionais.
Cada entrega acrescenta novas capacidades.
O produto torna-se mais completo.

COMBINAÇÃO: melhorar o que existe + acrescentar o que falta

Iteração e incremento representam dimensões diferentes

Iteração: “Como melhorar?”

Refinamento do produto existente.
Exemplos: usabilidade, desempenho, regras, arquitetura e qualidade.

Incremento: “O que acrescentar?”

Ampliação funcional do produto.
Exemplos: novo módulo, funcionalidade, integração ou serviço.

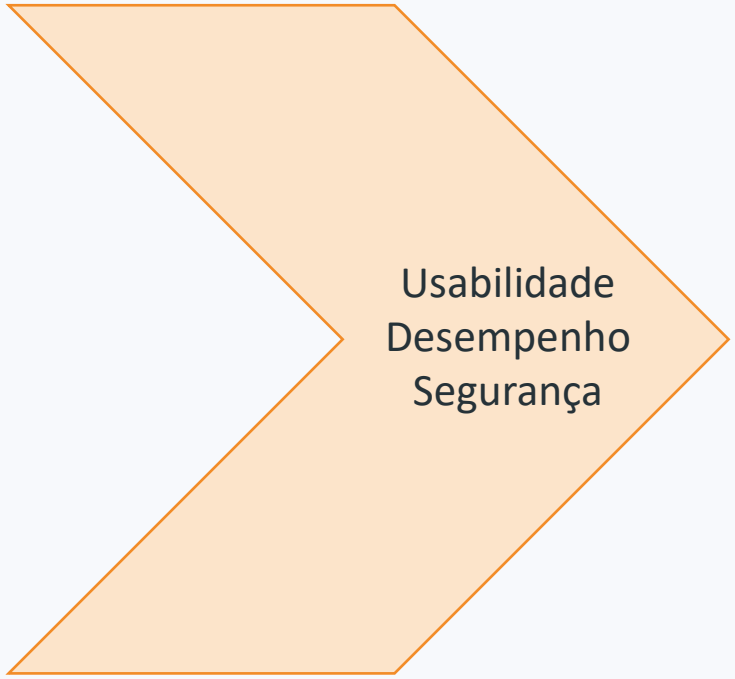
Podem existir separadamente, mas a combinação potencializa aprendizagem e valor.

Um sistema acadêmico evolui em duas dimensões

Incremento 1 — Autenticação e cadastro

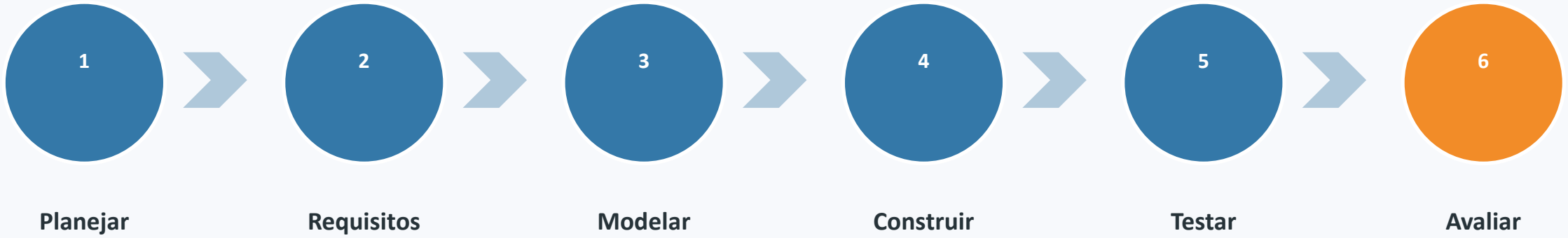
Incremento 2 — Matrícula em disciplinas

Incremento 3 — Lançamento de notas



O produto cresce por incrementos e amadurece por iterações.

Cada iteração percorre um ciclo completo



Resultado: versão integrada + feedback + prioridades atualizadas

O ciclo recomeça com base no que foi aprendido.

Vantagens: valor antecipado e redução de incerteza

Valor mais cedo

Partes prontas podem ser usadas antes do produto completo.

Resposta a mudanças

Feedback permite corrigir direção durante o projeto.

Problemas precoces

Falhas técnicas e de requisitos aparecem antes.

Testes frequentes

Cada incremento cria oportunidades de verificação.

Priorização por valor

Funcionalidades importantes podem vir primeiro.

Aprendizagem contínua

Produto e processo evoluem com evidências reais.

Desvantagens e riscos de aplicação

Gestão mais exigente

Iterações aumentam coordenação e decisões.

Escopo crescente

Feedback sem prioridade pode ampliar trabalho continuamente.

Arquitetura degradada

Incrementos sem visão estrutural elevam dívida técnica.

Integração complexa

Partes podem criar dependências difíceis.

Expectativas inadequadas

Versões iniciais podem parecer produto final.

Documentação insuficiente

Ritmo acelerado pode reduzir registros necessários.

Síntese: quando o modelo funciona melhor

MAIS ADEQUADO QUANDO

- ✓ Requisitos podem evoluir
- ✓ O produto pode ser dividido
- ✓ Há feedback frequente
- ✓ Existe integração contínua
- ✓ Valor antecipado importa

EXIGE CUIDADO QUANDO

- ! A arquitetura não suporta evolução
- ! Usuários não participam
- ! Não há priorização clara
- ! Dependências são excessivas
- ! O desenho precisa ser completo antes

Iterar gera aprendizagem. Incrementar gera valor. Integrar sustenta a evolução.

02

Modelo em espiral

Cada ciclo é organizado em torno da identificação e redução dos principais riscos.



03 | CONCEITO

O modelo espiral é dirigido pelos riscos

Cada volta produz aprendizado, evidências e compromissos.



Iterativo

A solução é revisitada em ciclos sucessivos.

Incremental

O produto ou o conhecimento evolui progressivamente.

Orientado a riscos

As atividades são escolhidas para reduzir incertezas críticas.

04 | CICLO

Quatro setores estruturam cada volta

A sequência apoia decisão, aprendizagem e compromisso progressivo.

1 Determinar objetivos

Metas, alternativas e restrições.

2 Avaliar alternativas e riscos

Identificar ameaças e opções de mitigação.

3 Desenvolver e testar

Gerar solução, protótipos e evidências.

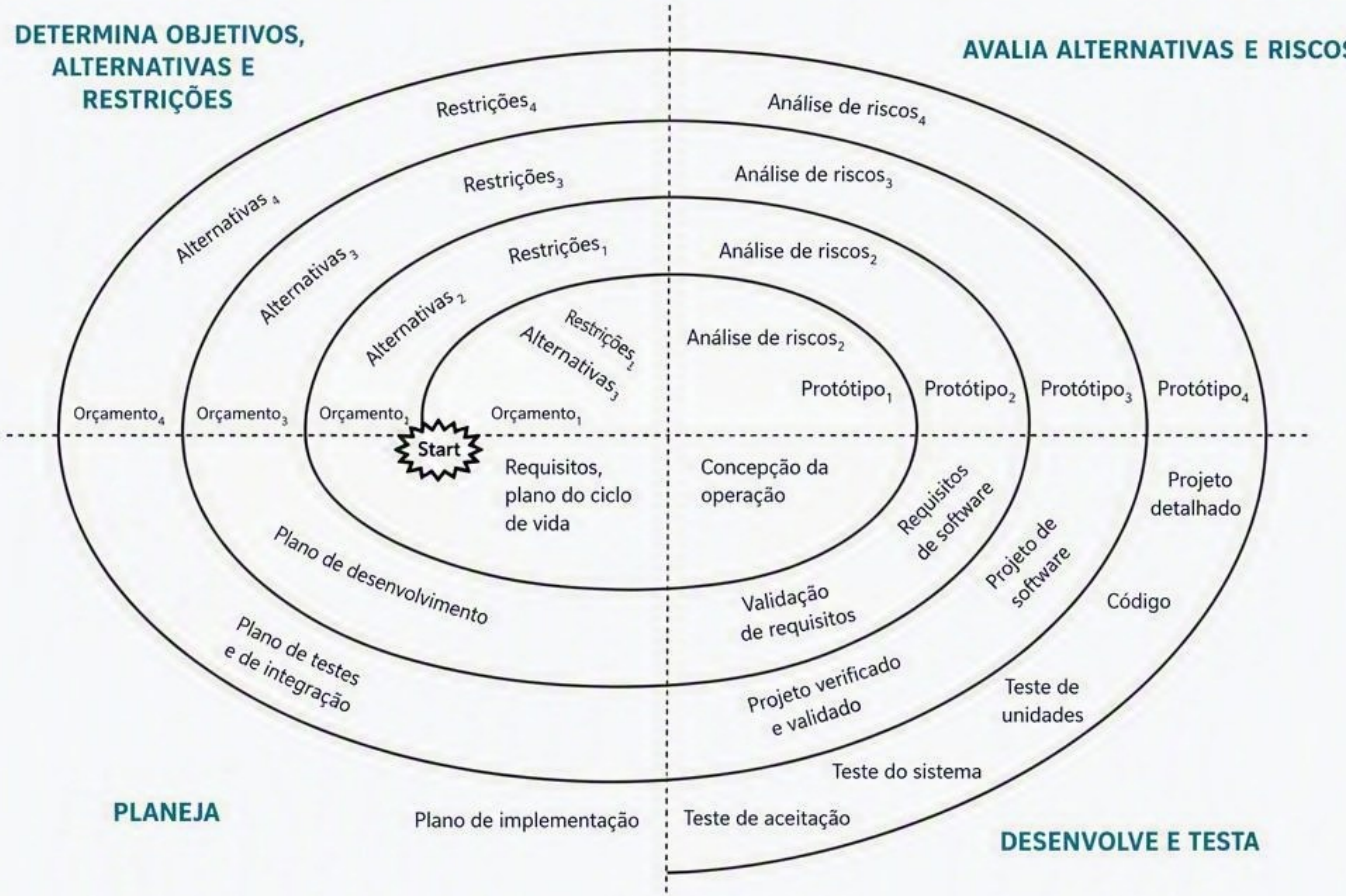
4 Planejar o próximo ciclo

Revisar resultados e decidir o próximo passo.

Cada ciclo termina com uma decisão: prosseguir, redirecionar ou interromper.

O Modelo Espiral – Visão completa

Cada volta da espiral representa um ciclo iterativo orientado por riscos.



Objetivo central: reduzir incertezas e riscos progressivamente, tomando decisões informadas em cada ciclo da espiral.

Como interpretar

1. Determina objetivos

Definir metas, alternativas de solução, restrições (tempo, custo, recursos) e estimativas de orçamento.

2. Avalia alternativas e riscos

Identificar, analisar e priorizar riscos, explorar alternativas e construir protótipos para reduzir incertezas.

3. Desenvolve e testa

Desenvolver a solução (código), integrar e testar, gerando incrementos funcionais e evidências.

4. Planeja o próximo ciclo

Revisar resultados, decidir o próximo passo, atualizar o plano e iniciar um novo ciclo da espiral.

Importante

O número de voltas é variável e depende da complexidade, dos riscos e do contexto do projeto.

Fonte: Adaptado de BOEHM, B. W. A Spiral Model of Software Development and Enhancement. IEEE Computer, v. 21, n. 5, p. 61-72, 1988.

Imagens: geradas com Microsoft Copilot

05 | RISCO

Prototipar é uma estratégia, não um fim

O protótipo deve responder à incerteza que ameaça a viabilidade.

Risco de requisitos

Protótipo navegável + teste com usuários

Risco técnico

Prova de conceito + ensaio de desempenho

Risco de segurança

Modelo de ameaças + experimento de arquitetura

Risco de adoção

Piloto controlado + coleta de feedback

Regra didática: prototipe para aprender, não apenas para demonstrar.

07 | AVALIAÇÃO

Vantagens do modelo espiral

Sua força aparece quando incerteza e retrabalho têm alto custo.

Risco antecipado

Investiga ameaças antes de grandes compromissos.

Flexibilidade

Adapta técnicas ao risco dominante.

Feedback contínuo

Valida resultados progressivamente.

Arquitetura por evidência

Testa decisões estruturais.

Compromisso progressivo

Amplia investimento conforme a viabilidade.

Integração de abordagens

Combina prototipação e incrementos.

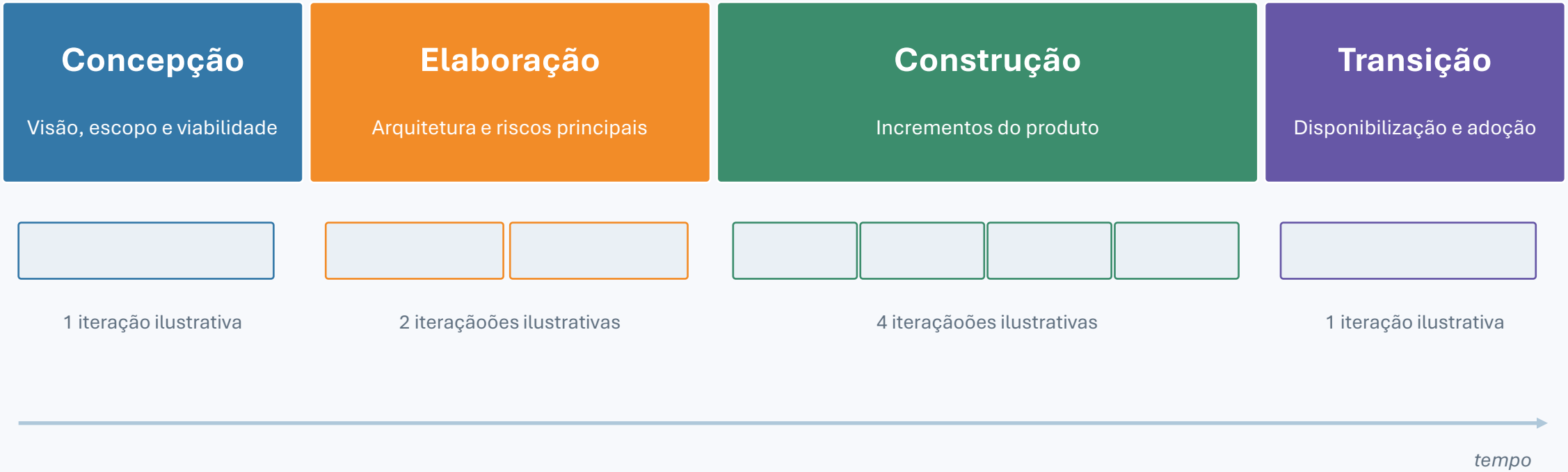
03

Processo Unificado

Dirigido por casos de uso, centrado na arquitetura e executado de forma iterativa e incremental.



Fases definem objetivos gerenciais, não etapas rígidas

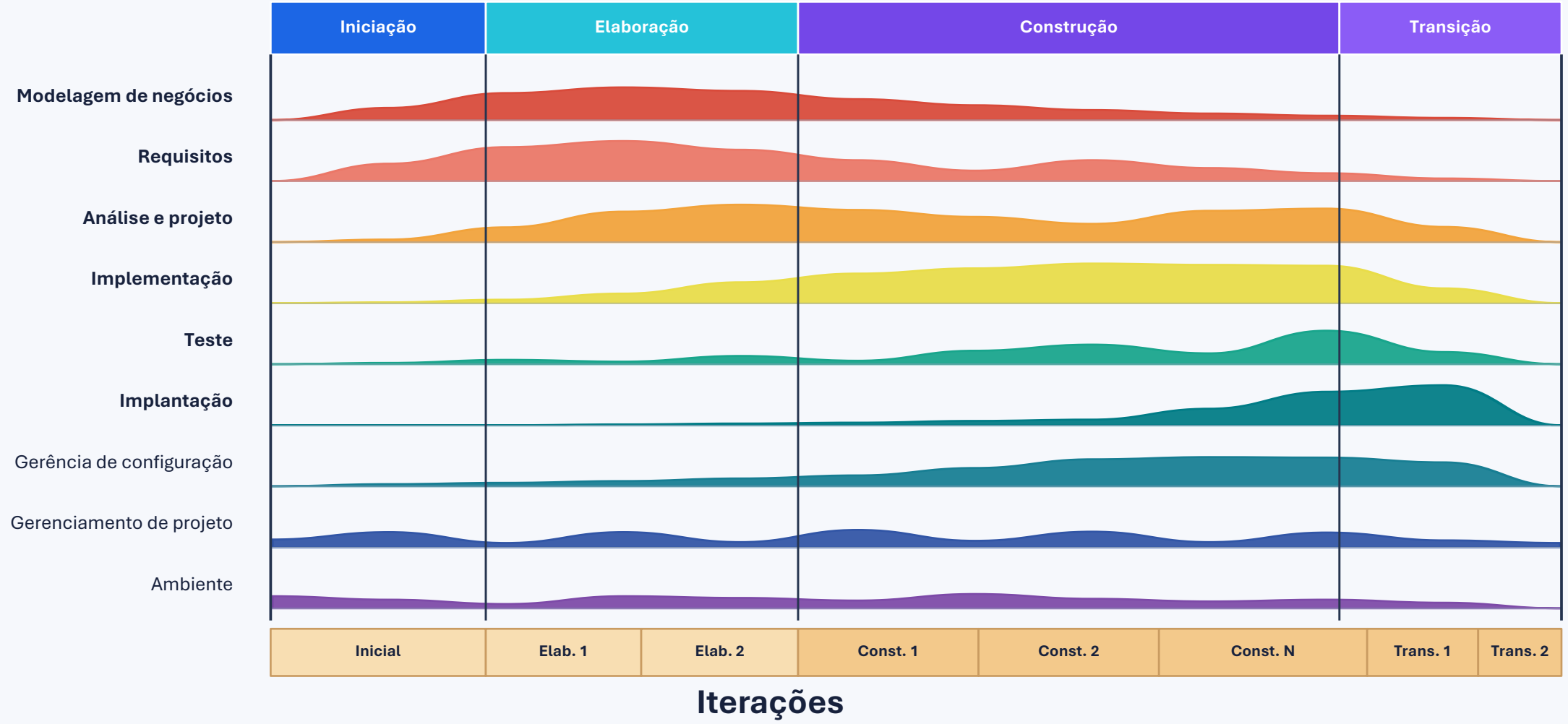


Uma fase pode conter várias iterações; cada iteração percorre as disciplinas necessárias.

VISÃO BIDIMENSIONAL

O “diagrama das baleias” do RUP

Áreas preenchidas formam silhuetas bem visíveis e representam a intensidade relativa de cada disciplina.

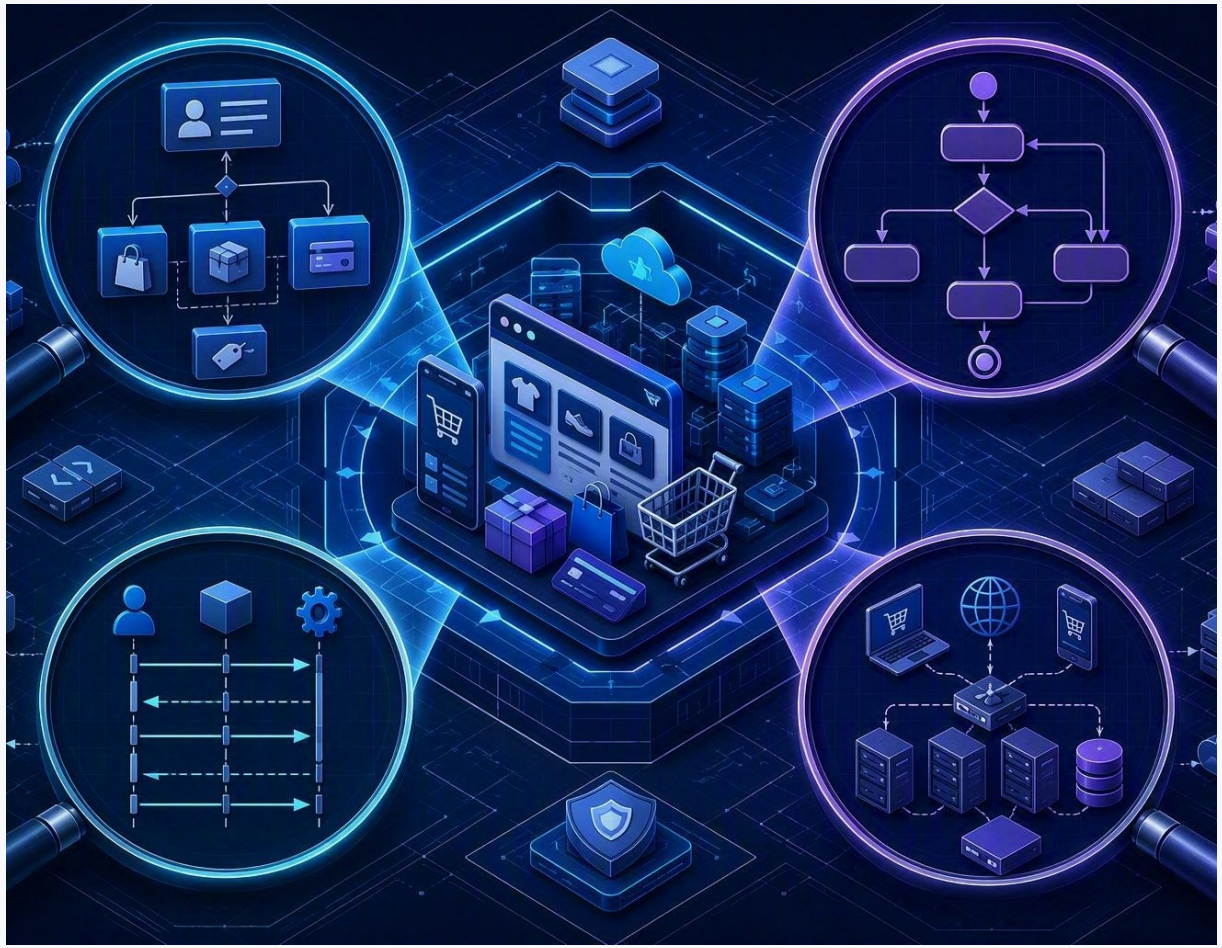


RUP organiza o caminho.
UML torna o sistema visível.

O diagrama das baleias conecta as duas dimensões:

- fases e iterações no tempo
- disciplinas com ênfases variáveis
- atividades sobrepostas, não sequenciais

UML: múltiplas perspectivas do mesmo sistema



Estrutura

Classes, objetos, componentes e implantação.

Comportamento

Casos de uso, estados e atividades.

Interação

Sequência, comunicação e colaboração.

Como o sistema é organizado

Classes

Classes, atributos, operações e relacionamentos.

Objetos

Instâncias com valores em determinado momento.

Componentes

Partes de software e dependências.

Implantação

Nós físicos e distribuição dos componentes.

Pergunta-guia: quais elementos existem e como estão conectados?

O que o sistema faz

Casos de uso

Funcionalidades vistas pelos atores externos.

Estado

Estados possíveis e transições do objeto.

Atividades

Fluxo de ações, controle ou dados.

Pergunta-guia: que comportamento precisa ser compreendido?

Como os participantes colaboram

Sequência

Enfatiza a ordem temporal das mensagens.

Comunicação

Enfatiza as conexões entre objetos.

Colaboração

Mostra objetos e mensagens para realizar uma tarefa.

Perspectivas diferentes podem representar o mesmo cenário.

Um e-commerce visto por diferentes diagramas



Do pedido à implantação

O mesmo domínio permite comparar funcionalidades, estrutura de dados, mensagens, estados do pedido, fluxo de compra e arquitetura física.

EXEMPLO 1

Classes e objetos: tipo versus instância

Cliente

nome: String
idade: int
cadastrar()
atualizar()

Associação

Um cliente pode realizar vários pedidos.

Pedido

data: Date
total: float
adicionarItem()
calcularTotal()

Objeto é uma ocorrência concreta de uma classe, com valores atribuídos.

EXEMPLO 2

Da necessidade à troca de mensagens

Casos de uso

Cliente visualiza produtos e realiza uma compra.

Sequência

Comprar → adicionar ao carrinho → finalizar → confirmar.

O caso de uso mostra o objetivo; a sequência detalha a colaboração temporal.

EXEMPLO 3

Estado e atividade: duas leituras

Estado do pedido

Em andamento → aguardando pagamento → pago → enviado.

Atividades da compra

Escolher → adicionar → finalizar → pagar → concluir.

Estado acompanha o objeto; atividade acompanha o fluxo do processo.

EXEMPLO 4

Componentes e implantação

Aplicação Web

Frontend e backend.

Servidor Web

Apache HTTPD e aplicação.

Banco de Dados

MySQL Server.

Componente descreve a parte de software. Implantação descreve onde ela executa.

DECISÃO DE MODELAGEM

Qual diagrama usar?

Escolha pela pergunta que precisa ser respondida.

Pergunta	Diagrama
Quem usa e para quê?	Casos de uso
Quais conceitos e relações?	Classes
Em que ordem interagem?	Sequência
Como o objeto muda?	Estado
Qual é o fluxo?	Atividades
Onde executa?	Implantação

Escolha o modelo pelo problema dominante

Iterativo e incremental

FOCO

Valor e aprendizagem

CENÁRIO MAIS ADEQUADO

Requisitos evolutivos

Espiral

FOCO

Risco e experimentação

CENÁRIO MAIS ADEQUADO

Alta incerteza

UP / RUP

FOCO

Arquitetura e disciplina

CENÁRIO MAIS ADEQUADO

Governança e rastreabilidade

Iteração gera aprendizagem. Incremento gera valor. A espiral reduz riscos. O UP estrutura a execução.

11 | SÍNTESE

Qual abordagem escolher?

A decisão depende do risco e da capacidade de governança.

Inovação + risco técnico alto | ESPIRAL

Explorar alternativas e produzir evidências.

Processo organizacional robusto | RUP

Estruturar papéis, disciplinas, fases e marcos.

Alta criticidade + padronização | COMBINAR

Aplicar a lógica de risco em um RUP configurado.

Para debate: quando o custo de uma decisão errada justifica um ciclo adicional?

REFERÊNCIAS

Fontes utilizadas nas apresentações

Unidade 2: Modelos de Processos de Software

1. Iterativo e Incremental

- LARMAN, C.; BASILI, V. R. Iterative and Incremental Development: A Brief History. IEEE Computer, 2003.
- SOMMERVILLE, I. Software Engineering. 10. ed. Pearson, 2015.
- PRESSMAN, R. S.; MAXIM, B. R. Software Engineering: A Practitioner's Approach. 9. ed. McGraw-Hill, 2019.
- Material-base: "6_ES - Modelo iterativo e incremental.pptx".

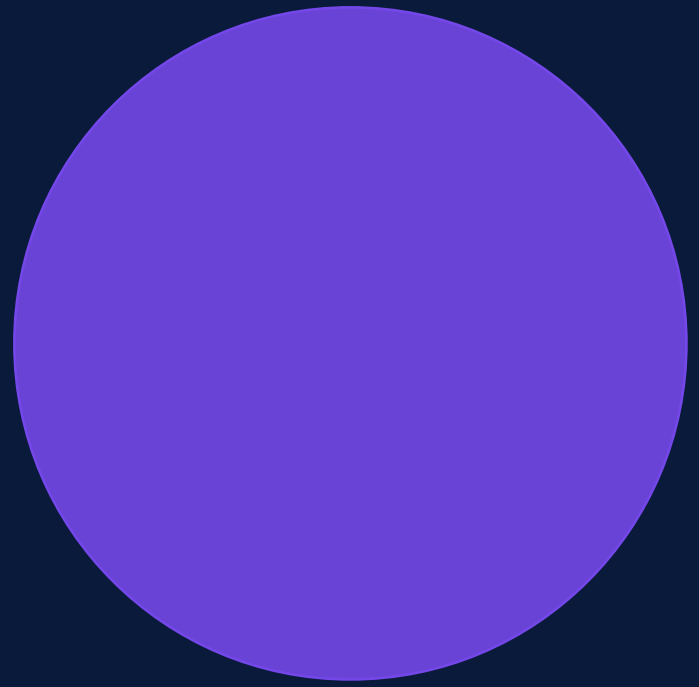
2. Modelo em Espiral

- BOEHM, B. W. A Spiral Model of Software Development and Enhancement. IEEE Computer, 1988.
- BOEHM, B. W. Spiral Development: Experience, Principles, and Refinements. 2000.
- SOMMERVILLE, I. Software Engineering. 10. ed. Pearson, 2015.
- Comparativo apoiado nos materiais internos sobre RUP e desenvolvimento iterativo.

3. RUP e UML

- KRUCHTEN, P. The Rational Unified Process: An Introduction. 3. ed. Addison-Wesley, 2003.
- JACOBSON, I.; BOOCH, G.; RUMBAUGH, J. The Unified Software Development Process. Addison-Wesley, 1999.
- OBJECT MANAGEMENT GROUP. OMG Unified Modeling Language, Version 2.5.1, 2017.
- Material-base: "6_ES - RUP.docx"; diagrama das baleias apoiado no RUP hump chart.

Observação: as ilustrações de apoio foram geradas com Microsoft Copilot; a referência visual do diagrama das baleias foi fornecida pelo autor.



Perguntas?